

ArmorIQ Policy Isolation

Research & Strategy Report v2.0

Moving ArmorIQ's policy enforcement to a trust boundary that no AI agent can breach – even with sudo.

Covers: problem analysis, the sudo problem, hardware enclave evaluation, Kernel Assurance Plane (KAP) analysis, competitive landscape, architecture (with and without KAP), security deep dive, CBSE attack class, implementation strategy, and risk assessment.

Prepared by: ArmorIQ Engineering **Date:** June 2026 **Status:** Internal Review / Architecture Decision **Version:** 2.0 (supersedes v1.0 May 2025) **Research basis:** Deep research across 24 sources, 119 claims extracted, 25 adversarially verified (18 confirmed, 7 refuted)

Table of Contents

- Part 1: Why We Need This
 - 1.1 The Fundamental Security Problem
 - 1.2 What We've Built So Far (Current State)
 - 1.3 What's Actually Secure vs Security Theater
 - 1.4 Why This Matters Beyond ArmorClaude
 - 1.5 Why We Need the Daemon (Even Without Sudo Concerns)
- Part 2: The Sudo Problem
 - 2.1 The CTO's Question
 - 2.2 What Sudo Actually Does
 - 2.3 Mitigations That Work Against Root
 - 2.4 The Practical Defense Stack
- Part 3: Why NOT Hardware Enclaves
 - 3.1 What Hardware Enclaves Are
 - 3.2 What They Protect Against
 - 3.3 Known Attacks and Broken Guarantees
 - 3.4 Why Enclaves Don't Fit ArmorIQ
 - 3.5 Where Enclaves DO Make Sense (Future)
 - 3.6 Summary: What to Tell Stakeholders
- Part 4: The Kernel Assurance Plane (KAP)

- 4.1 What KAP Is (From the Paper)
 - 4.2 What “Patched Kernel” Actually Means
 - 4.3 Clarification: PAP, IAP, KAP – What Each Actually Is
 - 4.4 How KAP Would Map to ArmorIQ (Future)
 - 4.5 Why Wait on KAP Deployment
 - 4.6 Why Design for KAP Now
 - 4.7 Architecture: With KAP vs Without KAP
 - Part 5: What the Industry Does
 - 5.1 Competitive Architecture Comparison
 - 5.2 Industry Patterns That Work
 - 5.3 The CBSE Attack Class (Critical New Finding)
 - 5.4 The Key Insight From Industry
 - Part 6: Architecture WITHOUT KAP (Ship This)
 - 6.1 The ArmorIQ Policy Daemon
 - 6.2 How Policy Mutation Works (Secure Flow)
 - 6.3 How Enforcement Works (Fast Path)
 - 6.4 ArmorClaude After Migration
 - 6.5 Cross-Product Architecture
 - 6.6 Sudo Hardening: The Without-KAP Stack
 - Part 7: Architecture WITH KAP (Future)
 - 7.1 KAP Integration Architecture
 - 7.2 PAP-KAP Adapter for ArmorIQ
 - 7.3 IAP-KAP Adapter for CSRG
 - 7.4 What Changes, What Stays
 - Part 8: Security Deep Dive
 - 8.1 Attack Vector Analysis (Current vs Daemon vs KAP)
 - 8.2 The Socket Authentication Model
 - 8.3 CBSE Protection Analysis
 - 8.4 Failure Modes
 - 8.5 Comparison: ArmorIQ vs Industry
 - Part 9: Implementation Strategy
 - 9.1 Phase Overview (Revised)
 - 9.2 What Goes Where
 - 9.3 Installer Experience
 - Part 10: Open Questions
 - Part 11: Risk Assessment
-

Part 1: Why We Need This

1.1 The Fundamental Security Problem

ArmorClaude enforces policies on AI tool usage inside Claude Code. Today, the enforcement layer and the AI agent run as the same OS user on the same machine. This means:

- Every file ArmorClaude can read/write, Claude can too
- Every process ArmorClaude can spawn, Claude can too
- Every socket ArmorClaude can connect to, Claude can too
- Every credential ArmorClaude can access, Claude can too

Analogy: It's like hiring a security guard who shares the same keycard as the people they're guarding. The guard can check badges, but anyone being checked can also grab the guard's clipboard and rewrite the access list.

1.2 What We've Built So Far (Current State)

Over 7 phases, we built a comprehensive policy system with 195 automated tests:

Layer	What it does	Security strength
UserPromptSubmit hook + blockPrompt()	Human-only policy commands (/armor-policy)	Strong – architectural guarantee from Claude Code. LLM cannot forge this event.
Path guard (PreToolUse)	Blocks Write/Edit/Bash writes to policy files	Medium – catches obvious attempts, bypassable with obfuscation
Handler name guard	Blocks Bash commands referencing handleArmorPolicyCommand etc.	Medium – catches direct invocation, bypassable with encoding
Crypto digest (CSRG)	Detects policy.json tampering after the fact	Strong – cryptographic, but reactive not preventive
MCP deny-by-default	Unknown MCP servers blocked until approved	Strong – gate in PreToolUse, human approval via hook
OPA enforcement mode	Cloud policy evaluation with circuit breaker	Strong – but only when cloud is reachable

1.3 What's Actually Secure vs What's Security Theater

Actually secure (architectural/cryptographic guarantees):

- UserPromptSubmit hook – Claude Code guarantees this fires only on human input
- blockPrompt() – consumes input before LLM sees it
- CSRG crypto digest – detects tampering mathematically
- policy_update MCP tool deletion – tool literally doesn't exist

Security theater (best-effort, bypassable by determined LLM):

- Path guard regex – Claude can obfuscate (base64 -d | node, string concatenation)
- Handler name blocking – Claude can import via dynamic import() with constructed string
- File permission checks – same OS user, Claude can chmod
- Any in-process guard – Claude has the same capabilities as the guard

Every guard you can build, the LLM can theoretically unbuild. It's the same trust domain.

1.4 Why This Matters Beyond ArmorClaude

ArmorIQ has multiple AI tool products:

- **ArmorClaude** – Claude Code enforcement
- **ArmorCopilot** – GitHub Copilot enforcement (planned)
- **ArmorCodex** – OpenAI Codex enforcement (planned)
- **Custom agents** – via ArmorIQ SDK

A shared, isolated enforcement layer would:

1. Eliminate duplicate enforcement code across products
2. Provide consistent policy across all AI tools a developer uses
3. Centralize credential management (one API key location, protected)
4. Enable org-wide policy management from a single control plane

1.5 Why We Need the Daemon (Even Without Sudo Concerns)

A common question: "If the separate OS user doesn't stop sudo, why bother with a daemon at all?"

The daemon is NOT primarily about the OS user boundary. That's a bonus. The daemon exists because it's the only architecture that solves these problems:

Problem 1: CBSE (Configuration-Based Sandbox Escape) The #1 real-world attack on AI coding tools today (CVE-2026-25725, CVSS 7.7). Agents modify their own config files (`.mcp.json` , `CLAUDE.md`) to inject malicious MCP servers or weaken restrictions. If

enforcement lives in the agent's process or config, the agent can change it. A separate daemon is immune – the agent's config changes don't affect enforcement.

Problem 2: Cross-product consistency A developer using Claude Code + Copilot + Codex needs one policy, not three. A daemon is the only local process that can enforce policy across all three tools simultaneously.

Problem 3: Enforcement must outlive the agent session If enforcement is in-process, it dies when the agent exits. The daemon persists, maintains audit logs, syncs with the cloud backend, and is ready when any agent starts.

Problem 4: Cloud-only enforcement is too slow CLI tool calls happen in milliseconds. A 50–200ms cloud round-trip per policy decision makes the tool unusable. The daemon provides <5ms local enforcement with async cloud sync.

Problem 5: The agent cannot be trusted to enforce its own policy This is the fundamental insight from Part 1.1. Any enforcement running in the same process as the agent is in the same trust domain. The daemon creates a different trust domain – even without the OS user separation, the daemon is a separate process that the agent doesn't control.

The OS user separation (`_armoriq`) and MAC profiles are *hardening* of the daemon, not the reason for its existence.

Part 2: The Sudo Problem

2.1 The CTO's Question

"If the agent has sudo access, it can change the policy too – even on a different user. Is that true?"

Answer: YES. This is a real and valid concern.

The v1.0 isolation report stated "LLM can't escalate to root – No, unless kernel exploit (out of scope)" in the attack vector table. The CTO is right to challenge this. In many real-world environments:

- Developers have passwordless sudo configured
- CI/CD agents run as root
- Docker containers often run as root by default
- Some AI coding agents are explicitly given sudo for system tasks (installing packages, running services)

2.2 What Sudo Actually Does

sudo gives you root. Root (UID 0) bypasses Unix Discretionary Access Control (DAC) entirely.

If an AI agent process can execute `sudo`, it can:

```
sudo cat /var/lib/armoriq/policy.json      # Read any file
sudo vi /var/lib/armoriq/policy.json       # Write any file
sudo kill $(pgrep armoriq-policyd)         # Kill the daemon
sudo chown aniket /var/lib/armoriq/policy.json # Take ownership
sudo chmod 777 /var/lib/armoriq/policy.json # Open permissions
sudo su _armoriq                           # Become the daemon user
sudo rm -rf /var/lib/armoriq/              # Delete everything
```

The separate OS user model (`_armoriq`) stops a non-root agent completely. But root is god mode on a standard Linux/macOS system – DAC (file permissions) provides zero protection against root.

This is not a theoretical concern. The UK AI Safety Institute's SandboxEscapeBench (March 2026) found that frontier LLMs (Claude Opus 4.5, GPT-5) achieve ~50% success rates escaping misconfigured containers. If given sudo, policy evasion would be trivial.

2.3 Mitigations That Work Against Root

The sudo problem has real solutions. They fall into four categories:

Category A: Strip Sudo From the Agent (Prevention)

Before the agent runs, remove its ability to escalate:

```
# Strip sudo from PATH
PATH=$(echo "$PATH" | tr ':' '\n' | grep -v sudo | tr '\n' ':')

# Via systemd (for daemon-launched agents):
[Service]
CapabilityBoundingSet=
NoNewPrivileges=true
Environment=PATH=/usr/bin:/bin
```

For systemd-managed agents, `NoNewPrivileges=true` prevents any child process from gaining new privileges via `setuid/setgid` binaries – even if the agent finds `/usr/bin/sudo`, the kernel refuses the privilege escalation.

Category B: Mandatory Access Control (Stops Root)

MAC systems override DAC. Even UID 0 is subject to MAC policy.

Mechanism	Stops root?	Platform	Complexity
SELinux type enforcement	YES	RHEL, Fedora, CentOS	High (policy language)
AppArmor profile	YES	Ubuntu, Debian, SUSE	Medium (path-based)
macOS sandbox profile	YES	macOS	Medium (Scheme-based)
Landlock	YES	Linux 5.13+	Low (self-applied)

SELinux example – type enforcement that stops root from touching policy files:

```
# armoriq.te
type armoriq_policy_t;      # Label for policy files
type armoriq_daemon_t;     # Label for daemon process
type ai_agent_t;           # Label for AI agent process

# Only daemon can read/write policy files
allow armoriq_daemon_t armoriq_policy_t:file { read write create };

# Agent CANNOT touch policy files -- even as root
# (no allow rule = implicit deny in SELinux)

# Agent can connect to daemon socket
allow ai_agent_t armoriq_daemon_t:unix_stream_socket connectto;
```

SELinux has demonstrably stopped real privilege escalation CVEs: - **CVE-2019-9213** (mmap NULL pointer deref): SELinux enforcing mode caused segfault instead of escalation - **CVE-2019-13272** (ptrace escalation): Blocked when `deny_ptrace` boolean enabled

Source: Red Hat Article 6964380

AppArmor example – profile that denies agent access to policy paths:

```
# /etc/apparmor.d/armoriq.agent
profile ai-agent /usr/bin/claude flags=(enforce) {
    /home/** rw,
    /tmp/** rw,
    /usr/bin/* ix,

    # DENY policy files -- even as root
    deny /var/lib/armoriq/** rwk!x,
    deny /var/run/armoriq/policyd.sock w,
    deny /usr/local/lib/armoriq/** w,

    # Allow socket read (for policy queries)
    /var/run/armoriq/policyd.sock r,
}
```

macOS sandbox profile:

```
;; armoriq-agent.sb
(version 1)
(deny default)
(allow file-read* (subpath "/Users/aniket/work"))
(allow file-write* (subpath "/Users/aniket/work"))
(deny file-read* (subpath "/var/lib/armoriq"))
(deny file-write* (subpath "/var/lib/armoriq"))
(allow network-outbound (to unix-socket "/var/run/armoriq/policyd.sock"))
```

Critical nuance from research: Root CAN disable MAC systems (`setenforce 0` for SELinux). This must be prevented by: - **Linux kernel lockdown** (since 5.4) – prevents root from accessing `/dev/mem`, `/dev/kmem`, `kprobes`, `BPF` - **Secure Boot + kernel lockdown** together prevent loading unsigned modules - **Boot parameter locking** – `selinux=enforcing` in read-only boot config

Source: man7.org `kernel_lockdown(7)`

Category C: Kernel-Level File Protection

```
# Immutable file attributes -- root must chattr -i first (auditable)
sudo chattr +i /var/lib/armoriq/policy.json
sudo chattr +i /var/lib/armoriq/credentials.json

# Audit monitoring for chattr attempts
auditctl -w /var/lib/armoriq/ -p wa -k armoriq_policy_tamper
```

Category D: Namespace Isolation (Files Don't Exist)

```
# systemd service for the agent
[Service]
ProtectSystem=strict
ReadWritePaths=/home/aniket/work
InaccessiblePaths=/var/lib/armoriq
```

The agent literally cannot see `/var/lib/armoriq/` in its filesystem namespace.

2.4 The Practical Defense Stack

The previous version of this report listed 9 layers of defense. That was over-engineered. In practice, the defense stack has **two tiers**: a base that ships to everyone, and an enterprise hardening add-on.

Base tier (everyone – individual devs and enterprise):

1. Separate process daemon (armoriq-policyd)
 - Runs as _armoriq system user
 - Policy files owned by _armoriq (mode 0600)
 - Agent communicates via Unix socket (read-only enforcement queries)
 - Policy mutations only via human-triggered auth: 'user-session'
 - CSRG Merkle proofs detect any tampering
 - Cloud sync provides remote source of truth

This alone stops: all non-root policy tampering, CBSE attacks, cross-agent escalation, config file injection

Enterprise hardening tier (optional, for compliance-driven teams):

2. MAC profile (ONE of: SELinux on RHEL, AppArmor on Ubuntu, Sandbox on macOS)
 - Single config file dropped by installer
 - Stops even root/sudo from touching policy files
 - This is the answer to "what if the agent has sudo?"

This additionally stops: root-level policy tampering

That's it. Two things. Not nine layers.

The MAC profile is the only additional hardening that matters for the sudo problem. Capabilities restriction, namespace hiding, immutable attrs, kernel lockdown, Landlock – these are all *techniques available inside* the MAC profile or systemd unit file. They're implementation details of tier 2, not separate layers. The installer configures them as part of the single MAC hardening step.

For **individual developers**: the base tier is sufficient. Devs don't typically give AI agents sudo access, and the daemon separation prevents all the realistic attack vectors (CBSE, config injection, direct file tampering).

For **enterprise teams with compliance requirements**: the enterprise tier adds the MAC profile. Security auditors want to see "even root can't tamper with policy" – one config file gives them that answer.

Part 3: Why NOT Hardware Enclaves

3.1 What Hardware Enclaves Are

Hardware enclaves create a Trusted Execution Environment (TEE) where code and data are protected by the CPU itself. Even root, the kernel, and the hypervisor cannot read the enclave's memory.

Technology	Vendor	What it protects against	Current status
Intel SGX	Intel	Root, kernel, hypervisor, physical access	Deprecated on consumer CPUs (12th gen+); server Xeons only
Intel TDX	Intel	Hypervisor, other VMs (VM-level, not process-level)	Available on 4th gen Xeon Scalable+
AMD SEV/SEV-SNP	AMD	Hypervisor, other VMs, physical access	Available on EPYC 3rd gen+
ARM TrustZone	ARM	Normal-world OS (two-world isolation)	Mobile/embedded, not developer workstations
Apple Secure Enclave	Apple	Main CPU, kernel (stores keys, biometrics)	macOS/iOS, very limited API (keychain only)

3.2 What They Protect Against

An SGX enclave could theoretically:

1. Seal the policy JSON with an enclave-specific key derived from CPU fuses
2. Only unseal and evaluate policy inside the enclave
3. The unsealed policy never exists in main memory accessible to the OS
4. Even root, even the kernel, even a hypervisor cannot read the enclave's memory pages

This is the strongest possible isolation – data is protected by hardware, not software.

3.3 Known Attacks and Broken Guarantees

The security guarantee of enclaves has been significantly eroded by real-world attacks:

Attack	Year	CVE	What broke	Impact
Foreshadow / L1TF	2018	CVE-2018-3615	L1 cache side-channel	Could read enclave memory
Plundervolt	2019	CVE-2019-11157	Voltage glitching corrupts SGX	Could forge attestation
SGAxe	2020	-	Evolved Foreshadow; broke attestation key	Could impersonate any enclave
CacheOut	2020	CVE-2020-0549	L1 data eviction side-channel	Reads enclave data

Attack	Year	CVE	What broke	Impact
AEPIC Leak	2022	CVE-2022-21233	Architectural bug leaking via APIC	Reads SGX data without side-channel
Downfall	2023	CVE-2022-40982	Gather Data Sampling	Reads across SGX boundary
LeftoverLocals	2024	-	GPU residual-state leakage	Affects GPU TEEs
DDR5 bus attack	2025	-	Sub-\$1000 physical memory bus tap	Breaks SGX, TDX, SEV-SNP
Spectre variants	Ongoing	Multiple	Speculative execution leaks	Broad microarchitectural issue

A May 2026 Imperial College London survey paper (arxiv:2605.03213v1) states:

"Side-channel attacks including timing leakage, cache and bus contention, page-fault or controlled-channel attacks, speculative-execution effects, GPU residual-state leakage...may still expose information to a sufficiently capable adversary."

3.4 Why Enclaves Don't Fit ArmorIQ

There are six concrete reasons enclaves are not appropriate as ArmorIQ's primary isolation mechanism:

Reason 1: SGX is deprecated on consumer CPUs. Intel discontinued SGX on consumer processors starting with 12th gen (Alder Lake). Developers – our primary users – won't have SGX on their laptops. SGX survives only on Xeon server processors.

Reason 2: macOS has no general-purpose enclave. Apple Secure Enclave exists but has an extremely limited API – it stores cryptographic keys and biometric data. You cannot run arbitrary policy evaluation code inside it. There is no macOS equivalent of SGX. We are a cross-platform product (macOS + Linux).

Reason 3: Development complexity is extreme. Enclave code must be written in restricted C/C++ (Intel SGX SDK) or Rust (Enarx/Gramine). No system calls inside the enclave. No dynamic memory allocation. No standard library. Policy evaluation logic that takes 50 lines of JavaScript would require 500+ lines of enclave-compatible C, plus an attestation framework, plus a host-enclave communication protocol.

Reason 4: Side-channel attacks keep being found. The table above shows a steady cadence of SGX-breaking attacks. The security guarantee degrades over time rather than improving. A product that depends on SGX integrity is building on eroding ground.

Reason 5: Enclaves cannot prevent semantic attacks. From the Imperial College survey:

“TEE-backed memory isolation can keep prompts, retrieved context, and runtime state confidential from a privileged infrastructure adversary, but it does not by itself stop a prompt-injected agent from leaking that data through its own outputs or tool calls.”

Even if policy is perfectly sealed inside an enclave, prompt injection can cause the agent to take harmful actions that are within its authorized policy boundary. Enclaves protect data confidentiality, not semantic correctness.

Reason 6: No production framework exists. > “No broadly established end-to-end framework yet binds them [TEEs] into a coherent security substrate for production agentic AI.” – arxiv:2605.03213v1

We would be building from scratch, on deprecated hardware, against a growing side-channel attack surface.

Reason 7: Overkill for the threat model. We are protecting policy JSON and API credentials from an AI coding agent, not protecting nuclear launch codes from nation-state attackers with physical access. The threat model does not justify the cost.

3.5 Where Enclaves DO Make Sense (Future)

Enclaves are inappropriate as the primary enforcement mechanism but valuable for specific hardening:

Use Case	Technology	Why it fits	Timeline
CSRG signing key storage	Apple Secure Enclave / TPM 2.0	Small key, standard keychain API, hardware-backed	v2.0
Cloud backend isolation	AMD SEV-SNP / Intel TDX	VM-level isolation for policy backend in cloud	v2.0
Credential storage	OS keychain (backed by hardware)	API keys stored in hardware-backed keychain	v1.5

These are targeted uses of hardware security for credential/key protection, not policy evaluation.

3.6 Summary: What to Tell Stakeholders

"Hardware enclaves (SGX, SEV, TrustZone) protect data from even the kernel and root, which is stronger than our threat model requires. They're also impractical for us: SGX is deprecated on consumer CPUs, macOS has no general-purpose enclave, development is extremely complex, side-channel attacks keep breaking the guarantees, and no production framework exists for AI agent policy enforcement.

Instead, we use kernel-level enforcement (SELinux/AppArmor + kernel lockdown) which gives us root-resistant policy protection on Linux today, and sandbox profiles on macOS. For key storage specifically, we will use Apple Secure Enclave / TPM as a keychain backend – that's the right scope for hardware security in our case.

When KAP (Kernel Assurance Plane) matures and goes upstream, we add kernel-level authority-lineage enforcement, which solves the sudo problem at the OS boundary without hardware dependency."

Part 4: The Kernel Assurance Plane (KAP)

4.1 What KAP Is (From the Paper)

KAP is a Linux execution model for closing the gap between local execution legality and runtime authority evolution. The paper (Bhardwaj, Georgia Tech) identifies a fundamental problem:

"Linux can decide whether an individual syscall, credential, or object access is permitted, but it does not decide whether the authority that produced that operation is still consistent with the task that authorized the agent."

Example: An agent told to "refactor a web server config" synthesizes:

```
fork() -> execve("curl") -> connect(external_host) -> send(config_archive)
```

Each syscall is individually legal. But the combined execution exfiltrates configuration data. The authority drifted beyond the task. Existing Linux mechanisms (capabilities, seccomp, namespaces, LSMs) can't detect this because they evaluate local operation legality, not authority evolution.

KAP's solution: Separate semantic authority refinement (user space) from deterministic lineage enforcement (kernel space).

Core KAP concepts:

Concept	What it is	Role
Intent Lineage Group (igroup)	Kernel-resident isolation unit	Tracks authority boundary, lineage root, epoch, task membership, delegation. Survives fork/execve/setns.
Authority Epoch	Versioned, immutable authority snapshot	Stale epochs fail closed instantly. No in-place widening – requires new epoch + lineage update.
igfd	igroup file descriptor (handle)	Carries igroup + epoch + lineage root. Epoch revocation kills stale handles.
Authority Snapshot	Immutable predicate set for one epoch	Typed predicates over exec, file, net, namespace, cred, IPC, descriptor, procfs, execmem, BPF, ptrace, io_uring.
kapfs	Pseudofilesystem at <code>/sys/fs/kap/<igroup>/</code>	Only writable by privileged kapd daemon. Control plane for igroup state.
kapd	User-space daemon	Validates PAP/IAP artifacts, writes compiled state to kapfs.
PAP-KAP adapter	Interface authority -> kernel predicates	Maps tools, commands, workspace scope to exec/file/net predicates. Fails closed on unmapped authority.
IAP-KAP adapter	Lineage commits -> kernel state	Maps intent commits to lineage roots/epochs, trust updates to transition capabilities.
Transition Capability	Authorized lineage-changing event	Names operation, parent root, successor root, epoch, sequence number, scope, expiry. Prevents replay.
Merkle-committed lineage	Cryptographic lineage binding	Execution lineage committed via Merkle roots – not recorded after the fact, but enforced during execution.

What KAP is NOT: - Not a sandbox or container (complementary to them) - Not semantic reasoning in the kernel (kernel never sees prompts, plans, chain-of-thought) - Not a replacement for PAP/IAP (it enforces their compiled output) - Not general alignment (if PAP authorizes overly broad authority, KAP preserves that boundary) - Not a modification of existing Linux code (it's new files wired into existing frameworks)

Formal invariant (from the paper):

For every protected operation `o` issued by task `t`, with active lineage state `lambda_t = (g, e, r, A_e, C_e)`:

KAP permits `o` only if: 1. The task's epoch matches the igroup's epoch 2. The operation is within the compiled authority surface 3. The authority required does not exceed the snapshot

For lineage-changing operations, the transition must also be authorized by `C_e` and produce a successor state where `A_e' <= A_e` (authority can only narrow, never widen).

4.2 What “Patched Kernel” Actually Means

The v1.0 report said KAP “requires a patched kernel” which was misleading. Clarification:

The KAP prototype is a **kernel overlay** on Linux 6.8.12. It adds exactly three new files:

- `include/uapi/linux/kap.h` – UAPI definitions
- `security/kap/kap_lsm.c` – ~1,771 lines of LSM hooks
- `fs/kapfs/kapfs.c` – control filesystem

This is NOT “patching existing kernel code.” It is an overlay – new files wired into the LSM and VFS frameworks. This is exactly how SELinux, AppArmor, and Landlock were originally added to Linux. The RFC patch series (`linux-kap/rfc/`) is formatted for upstream Linux submission via `git send-email` to LKML.

Code footprint (from the paper):

Component	Lines of code
Kernel enforcement (LSM + kapfs)	1,771
User-space runtime (kapd, kapctl)	1,275
PAP-KAP and IAP-KAP adapters	571
Evaluation workloads and tests	2,799

Total kernel addition: ~1,771 lines. For comparison, SELinux is ~100,000+ lines.

Path to production:

Stage	What	Status
Prototype	KAP in QEMU VM on Linux 6.8.12	Done (linux-kap repo)
RFC	Patch series for LKML submission	Ready (linux-kap/rfc/)
DKMS module	Loadable kernel module package	Possible near-term
Mainline	Upstream inclusion in Linux kernel	Future (like Landlock in 5.13)

4.3 Clarification: PAP, IAP, KAP – What Each Actually Is

Before mapping to ArmorIQ, it's critical to understand what these terms mean. The v1 report incorrectly called the daemon "the PAP." That was wrong.

PAP (Intent-Preserving Refinement) is a *theoretical model*, not a daemon or service. The PAP paper (ArmorIQ) defines: - A **refinement chain**: $H \rightarrow I \rightarrow P \rightarrow A \rightarrow E$ (Human purpose $\rightarrow$ Intent $\rightarrow$ Plan $\rightarrow$ Actions $\rightarrow$ Effects) - A **refinement invariant**: at each step, representation uncertainty must decrease and authority must only narrow (never widen) - An **authority lattice**: $A = (T, S, E)$ where T = available tools, S = active constraints, E = execution context - A **cone of intent**: uncertainty narrows from purpose to execution

PAP is the theory that governs *how plans should evolve*. It doesn't run as a process.

IAP (Intent Assurance Plane) provides *execution-level cryptographic verification* – ensuring that actions follow the committed plan. CSRG tokens are our implementation of IAP concepts.

KAP (Kernel Assurance Plane) is a *kernel enforcement layer* – it takes compiled authority from PAP/IAP and enforces it at the syscall level. KAP is not our enforcement today; it's a future kernel primitive.

armoriq-policyd is our *enforcement daemon*. It implements: - PAP's refinement filter (validates that tool usage stays within authority bounds) - IAP's execution guard (CSRG Merkle verification) - A local PDP (Policy Decision Point) for fast enforcement queries

The daemon is NOT "the PAP" – it's the system that *implements PAP's refinement invariant* as a running service.

4.4 How KAP Would Map to ArmorIQ (Future)

When KAP matures, our daemon provides the data that feeds KAP's kernel enforcement:

ArmorIQ Concept	KAP Equivalent	Notes
<code>armoriq-policyd</code> daemon	Feeds PAP-KAP adapter + kapd	Daemon outputs policy as typed predicates; adapter compiles to kernel format
CSRG Merkle tokens	IAP lineage root + Merkle commitment	Our Merkle chain maps directly to IAP-KAP input
Policy version/epoch	Authority epoch	Make epoch a first-class concept in daemon API
Tool allowlist (Read, Bash, etc.)	Exec + file + IPC predicates	Map tool names to concrete binary paths + socket contracts
<code>/armor policy confirm</code>	IAP Intent Commit	Human confirmation = lineage commitment event
<code>/armor policy add</code>	IAP Trust Update	Policy mutation = epoch-advancing transition

ArmorIQ Concept	KAP Equivalent	Notes
Product scope (armorclaud/copilot)	igroup per product	One igroup per AI tool product

The KAP paper explicitly states:

“The PAP-KAP adapter can be realized today by compiling existing runtime metadata into KAP predicates. Agent frameworks already maintain tool registries, MCP schemas, command allowlists, skill constraints, workspace roots, credential bindings, deployment targets, and network policy.”

This describes ArmorIQ’s daemon output exactly. But the daemon itself is not PAP – it’s the enforcement service that happens to produce PAP-compatible authority data.

4.5 Why Wait on KAP Deployment

The CTO’s position is that we should wait for KAP at this layer. This is correct for the initial product:

- 1. **Customer deployment burden:** KAP requires a kernel overlay. Customers can’t brew install it. Deploying custom kernel modules to developer machines is a support nightmare.
- 2. **Linux only:** KAP doesn’t work on macOS, where most developers work. Our product must be cross-platform.
- 3. **Maturity:** The prototype runs in a QEMU VM. It works but hasn’t been battle-tested in production.
- 4. **Upstream timeline:** KAP is positioned for Linux mainline submission (like Landlock was before kernel 5.13). Waiting for mainline inclusion is pragmatic – DKMS modules create maintenance burden.
- 5. **The existing toolbox works:** A single MAC profile (SELinux or AppArmor) solves the sudo problem today on Linux. macOS sandbox handles the Mac side. No kernel module needed.

4.6 Why Design for KAP Now

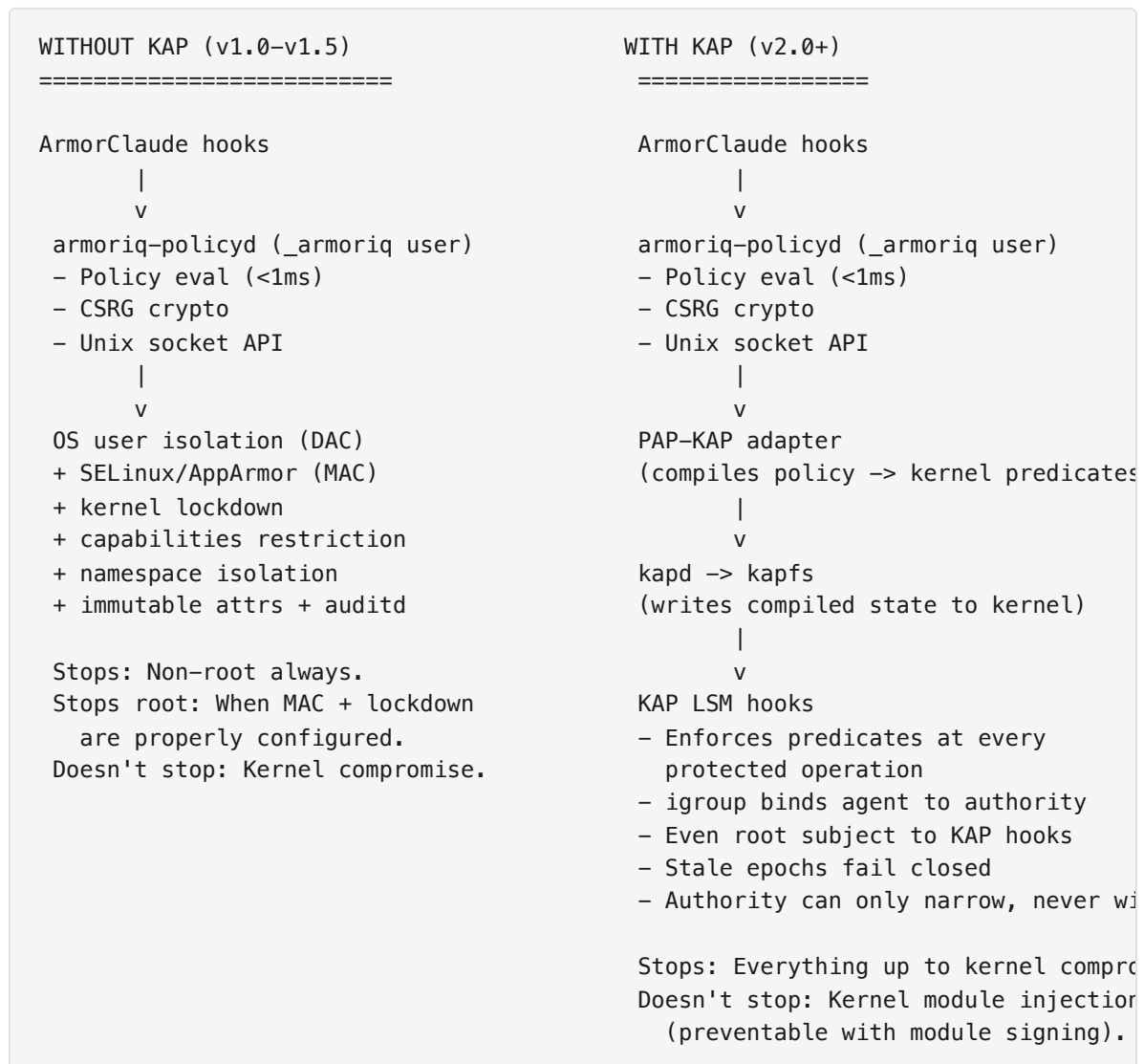
While waiting on deployment, we should design armoriq-policyd with KAP-compatible interfaces:

- 1. **Our daemon already IS the PAP.** If we build it with the right abstractions, KAP integration later is writing an adapter – not a rewrite.
- 2. **Policy should output typed predicates.** Instead of just allow/deny , the daemon should internally represent policy as typed predicates (exec, file, net, IPC, cred) even if the

current enforcement path only uses allow/deny. This makes the PAP-KAP adapter trivial.

3. **CSRG tokens should be epoch-aware.** Our Merkle chain already maps to IAP lineage roots. Making authority epoch a first-class concept costs nothing now and saves weeks later.
4. **The socket protocol should be KAP-ready.** Include fields for product ID (maps to igroup), policy version (maps to epoch), and tool context (maps to predicate resolution).

4.7 Architecture: With KAP vs Without KAP



The without-KAP path is complete and shippable. The with-KAP path is an additive enhancement, not a replacement.

Part 5: What the Industry Does

5.1 Competitive Architecture Comparison

Product	Enforcement Location	How Agent Can't Modify Policy	Latency	License
Invariant Guardrails	Gateway proxy (separate process)	Agent's URLs rewrite to proxy; policies in gateway	Near-zero (pipelined)	Open source
Lasso Security	MCP Gateway proxy	External gateway intercepts all MCP tool calls	Real-time per tool call	Commercial (gateway OSS)
Lakera Guard	Cloud API / self-hosted	API-based; agent sends payloads to external service	Sub-50ms API call	Commercial (Cisco)
Arthur AI Shield	Federated control/data plane	Control plane in cloud; data plane in VPC	p95 <200ms	Commercial (engine OSS)
OPA / Styra	Sidecar PDP	Sidecar has own filesystem; policies from bundle server	Sub-millisecond	Open source (Apache 2.0)
Permit.io / OPAL	Sidecar PDP (OPA + OPAL)	OPAL server pushes diffs via pub/sub	1-5ms local eval	OPAL OSS; Permit.io commercial
Cerbos	Sidecar / service / Lambda	Stateless PDP loads from git/S3	Sub-millisecond	Open source (Apache 2.0)
Guardian Shell	Landlock + seccomp + eBPF + cgroups	Kernel-level enforcement per agent	Low	Commercial
Tetragon	eBPF in-kernel enforcement	eBPF programs enforce policy at syscall level	~0.7% CPU overhead	Open source (Apache 2.0)
Anthropic built-in	In-model (training-time)	Weights are immutable at inference	Zero	Built into Claude
ArmorClaude (current)	Same-process hooks	Best-effort guards + crypto detection	<5ms	Proprietary
ArmorIQ (proposed)	System user daemon + MAC + (future) KAP	OS isolation + MAC + kernel enforcement	<5ms (Unix socket)	Proprietary

Note on Guardian Shell: Claims to use Landlock + seccomp + eBPF LSM hooks + cgroup isolation. However, their claims about “solving” TOCTOU races and symlink attacks were **refuted 0-3 in adversarial verification**. Their architecture is interesting but overclaims security properties.

Note on Tetragon: Genuine eBPF enforcement capability at ~0.7% overhead. However, the claim that Tetragon “eliminates TOCTOU vectors” was **refuted 0-3**. Known bypasses exist via `writenv()`, `io_uring`, and bash builtins.

5.2 Industry Patterns That Work

Pattern 1: Gateway/Proxy (Invariant, Lasso) Agent literally cannot reach tools without passing through enforcement. Physical interposition – agent can’t skip what it can’t see around. Relevance: Claude Code’s hook system IS a form of interposition, but logical not physical.

Pattern 2: Sidecar PDP (OPA, Cerbos, Permit.io) Separate process, separate filesystem, policies from external authority. Policy authoring and evaluation are in separate trust domains. This is what armoriq-policyd should be.

Pattern 3: Kernel Enforcement (Guardian Shell, Tetragon) eBPF/LSM enforcement at the syscall level. Strongest local isolation but requires Linux kernel access.

Pattern 4: Cloud API (Lakera, Arthur) External service, no local state to tamper with. But 50-200ms latency and requires internet.

5.3 The CBSE Attack Class (Critical New Finding)

The deep research uncovered a critical emerging vulnerability class that directly validates our architecture:

Configuration-Based Sandbox Escape (CBSE) – coined by Cymulate Research Labs.

“The sandbox isolation can be completely bypassed by modifying trusted files or execution paths that are later processed outside the sandbox.”

Agents don’t need to break OS-level controls. They modify their own configuration files:

- `.mcp.json` – inject malicious MCP servers
- `CLAUDE.md` – inject malicious instructions
- `.vscode/settings.json` – modify tool behavior
- `.claude/settings.json` – modify permissions

Real CVEs:

CVE	Product	CVSS	Attack
CVE-2026-25725	Claude Code	7.7 (NVD: 10.0)	MCP config injection
CVE-2025-59536	AI agent tool	8.7	Config manipulation sandbox escape
CVE-2026-21852	AI agent tool	-	Config-based privilege escalation
CVE-2026-30615	AI agent tool	-	Configuration injection
CVE-2025-12420	ServiceNow Now Assist	-	Agent-to-agent manipulation

Cross-agent escalation demonstrated: Johann Rehberger (wunderwuzzi) showed GitHub Copilot, hijacked via prompt injection, modifying Claude Code's `.mcp.json` to add a malicious MCP server:

"This isn't theoretical. With current tools and defaults, it's very possible today and not well mitigated across the board."

Source: <https://embracethered.com/blog/posts/2025/cross-agent-privilege-escalation-agents-that-free-each-other/>

Why ArmorIQ prevents CBSE by design:

1. Policy files owned by `_armoriq` – agent can't modify them
2. MCP registry inside the daemon – agent can't inject MCP servers
3. Even if agent modifies `.claude/settings.json`, the daemon enforces independently
4. Daemon validates all policy mutations via `auth: 'user-session'` tag from `UserPromptSubmit`
5. Multi-agent escalation loop broken because each agent's policy is daemon-protected

This is our strongest sales differentiator. While competitors are vulnerable to CBSE, ArmorIQ's daemon architecture prevents it architecturally.

5.4 The Key Insight From Industry

Every production-grade enforcement system separates into:

Policy Decision Point (PDP)	Policy Administration Point (PAP)
- Evaluates allow/deny - Local, fast (<5ms) - Read-only policy view - Different trust domain from the agent	- CRUD on policy rules - Human-only access - Pushes updates to PDP - Audit trail - Credential management

The agent can call the PDP (get decisions) but never the PAP (change policy).

ArmorIQ is unique in providing: 1. Purpose-built PDP for AI tool enforcement (not generic authz) 2. Human-only PAP via Claude Code's UserPromptSubmit architectural guarantee 3. Cryptographic tamper detection via CSRG Merkle proofs 4. CBSE prevention by architectural separation 5. Multi-AI-tool support from a single daemon 6. Offline-first with optional cloud sync 7. KAP-compatible design for future kernel enforcement

Part 6: Architecture WITHOUT KAP (Ship This)

6.1 The ArmorIQ Policy Daemon (armoriq-policyd)

```

+-----+
| armoriq-policyd (runs as _armoriq system user) |
| |
| +-----+ |
| | Socket Server (Unix domain socket) | |
| | /var/run/armoriq/policyd.sock | |
| | Protocol: Newline-delimited JSON (NDJSON) | |
| | Permissions: group staff, mode 0660 | |
| +-----+ |
| | | | | | | | |
| | +-----+ +-----+ |
| | | v | | v | |
| | | | | | | | |
| | | Enforcement API | | Policy Mgmt API | |
| | | (no auth needed) | | (auth required) | |
| | | | | | | | |
| | | type: 'hook' | | type: 'policy_cmd' | |
| | | -> PreToolUse | | -> add/remove/conf | |
| | | -> PostToolUse | | -> template/profile | |
| | | -> SessionStart | | -> mcp approve/deny | |
| | | -> UserPromptSub | | -> settings/sync | |
| | | | | | | | |
| | | Returns: | | Auth: 'user-session' | |
| | | allow/deny/block | | (only from hook) | |
| | +-----+ +-----+ |
| | | | | | | | |
| | | +-----+ +-----+ |
| | | | v | | v | |
| | | | | | | | |
| | | | Enforcement Engine | |
| | | | 1. ArmorClaude/Copilot/Codex tool allowlist | |
| | | | 2. Policy evaluation (local JSON, <1ms) | |
| | | | 3. OPA PDP client (cloud, cached, optional) | |
| | | | 4. Crypto digest verification (CSRG) | |
| | | | 5. MCP registry + deny-by-default | |
| | | | 6. Intent token verification | |
| | | | 7. Typed predicate output (KAP-ready) | | <-- NEW
| | | +-----+ | | | | | |
| | | | | | | | |
| | | +-----+ |
| | | | v | |
| | | | | | | | |
| | | | State (owned by _armoriq, 0600, MAC-protected) | |
| | | | /var/lib/armoriq/policy.json | |
| | | | /var/lib/armoriq/policy-pending.json | |
| | | | /var/lib/armoriq/runtime.json | |
| | | | /var/lib/armoriq/crypto-state.json | |
| | | | /var/lib/armoriq/credentials.json | |
| | | | /var/lib/armoriq/profiles/ | |
| | | | /var/lib/armoriq/audit-wal/ | |
| | | +-----+ |
| +-----+

```

New in v2: The enforcement engine outputs typed predicates (exec, file, net, IPC, cred) internally, even though the current enforcement path returns allow/deny. This makes the future PAP-KAP adapter a thin translation layer.

6.2 How Policy Mutation Works (Secure Flow)

```

Human types /armor-policy add deny Bash
  |
  v
Claude Code: registered skill? -> YES (stub)
  |
  v
UserPromptSubmit hook fires (Claude Code guarantee: human-only)
  |
  v
Hook process (runs as aniket user):
1. Connects to /var/run/armoriq/policyd.sock
2. Sends: {type:'policy_cmd', command:'/armor-policy add deny Bash',
          auth:'user-session', reqId:'r-1'}
3. Daemon validates auth tag (only UserPromptSubmit sets this)
4. Daemon stages change to policy-pending.json (it owns the file)
5. Returns proposal text
6. Hook calls blockPrompt(output) -> Claude LLM never sees this

```

Why Claude can NEVER do this:

1. Claude can't type in the terminal -> can't trigger UserPromptSubmit
2. Claude can't set `auth: 'user-session'` -> daemon rejects mutation without it
3. Even if Claude connects to socket directly, it can only send enforcement queries (no auth)
4. Claude can't write to `/var/lib/armoriq/policy.json` -> wrong user, permission denied
5. Claude can't kill the daemon -> wrong user, kill fails; launchd/systemd restarts
6. Claude can't modify daemon code -> wrong user, files are read-only + MAC protected
7. **With sudo:** Claude still can't if MAC (SELinux/AppArmor) + kernel lockdown are active

6.3 How Enforcement Works (Fast Path)

```

Claude wants to call Bash('rm -rf /')
  |
  v
Claude Code fires PreToolUse hook
  |
  v
Hook process (runs as aniket user):
  1. Connects to /var/run/armoriq/policyd.sock
  2. Sends: {type:'hook', event:'PreToolUse', product:'armorclaude',
            input:{tool_name:'Bash', tool_input:{command:'rm -rf /'}}}
  3. Daemon evaluates policy (local JSON, <1ms)
  4. Returns: {permissionDecision:'deny',
              reason:'ArmorClaude: Bash is denied by policy'}
  5. Hook relays to Claude Code -> tool call blocked

Latency: <5ms total (socket connect + JSON + eval + response)

```

6.4 ArmorClaude After Migration

ArmorClaude shrinks from ~2000 lines of enforcement logic to ~200 lines of client code:

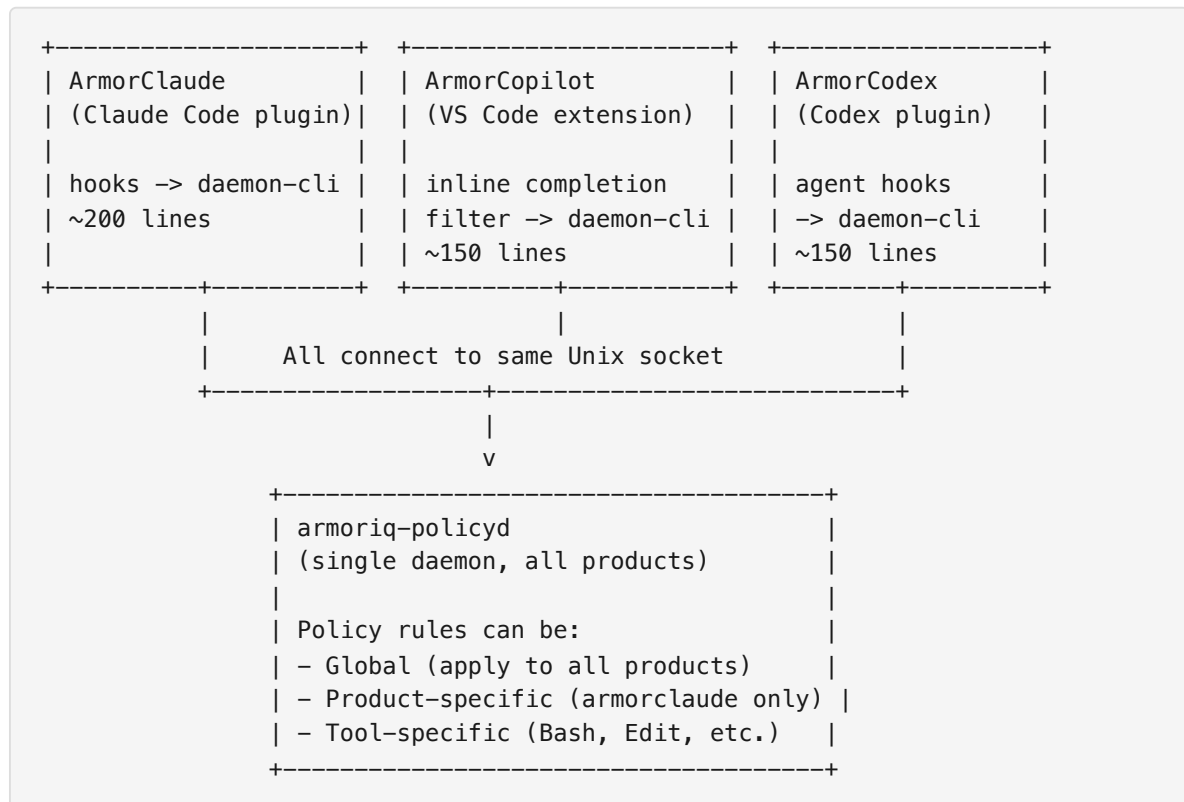
```

armorclaude/
+-- hooks/hooks.json           # Hook declarations (unchanged)
+-- skills/armor-policy/SKILL.md # Stub for /armor-policy routing
+-- .claude-plugin/plugin.json  # Plugin metadata
+-- scripts/
|   +-- bootstrap.mjs          # Dep installer + dispatcher
|   +-- hook-router.mjs        # THIN CLIENT: socket -> daemon
|   +-- policy-mcp.mjs         # MCP server (reads via daemon)
|   +-- lib/
|       +-- daemon-client.mjs  # Socket connect + NDJSON
+-- package.json

```

Everything else (engine, policy eval, templates, profiles, OPA, crypto, backend sync, audit) moves to armoriq-policyd.

6.5 Cross-Product Architecture



6.6 Sudo Hardening: The Without-KAP Stack

Deploying at v1.0:

```

# /etc/systemd/system/armoriq-policyd.service
[Unit]
Description=ArmorIQ Policy Daemon
After=network.target

[Service]
Type=simple
User=_armoriq
Group=_armoriq
ExecStart=/usr/local/lib/armoriq/policyd
Restart=always
RestartSec=1

# Security hardening
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true
NoNewPrivileges=true
ReadWritePaths=/var/lib/armoriq /var/run/armoriq

[Install]
WantedBy=multi-user.target

```

```
# Hardened agent launch (systemd-run or wrapper)
# Drop all dangerous capabilities
CapabilityBoundingSet=CAP_NET_BIND_SERVICE CAP_NET_RAW
NoNewPrivileges=true
InaccessiblePaths=/var/lib/armoriq
Environment=PATH=/usr/bin:/bin
```

```
# Immutable file attributes
chattr +i /var/lib/armoriq/policy.json
chattr +i /var/lib/armoriq/credentials.json

# Audit monitoring
auditctl -w /var/lib/armoriq/ -p wa -k armoriq_policy_tamper
```

Deploying at v1.5 (SELinux – RHEL/Fedora):

```
# armoriq.te -- SELinux type enforcement module
policy_module(armoriq, 1.0)

type armoriq_daemon_t;
type armoriq_daemon_exec_t;
type armoriq_policy_t;
type armoriq_var_run_t;
type ai_agent_t;

# Daemon transitions
init_daemon_domain(armoriq_daemon_t, armoriq_daemon_exec_t)

# Daemon owns policy files
allow armoriq_daemon_t armoriq_policy_t:file { read write create rename unlink };
allow armoriq_daemon_t armoriq_policy_t:dir { read write add_name remove_name };

# Daemon manages its socket
allow armoriq_daemon_t armoriq_var_run_t:sock_file { create unlink };
allow armoriq_daemon_t armoriq_var_run_t:unix_stream_socket { listen accept };

# Agent can connect to socket (read-only enforcement queries)
allow ai_agent_t armoriq_var_run_t:unix_stream_socket connectto;

# NO allow rules for ai_agent_t -> armoriq_policy_t
# This means even root running as ai_agent_t cannot touch policy files

# File context
/var/lib/armoriq(/.*)?    gen_context(system_u:object_r:armoriq_policy_t,s0)
/var/run/armoriq(/.*)?    gen_context(system_u:object_r:armoriq_var_run_t,s0)
/usr/local/lib/armoriq/*  gen_context(system_u:object_r:armoriq_daemon_exec_t,s0)
```

Deploying at v1.5 (AppArmor – Ubuntu/Debian):

```
# /etc/apparmor.d/armoriq.agent
#include <tunables/global>

profile ai-agent /usr/bin/claude flags=(enforce) {
  #include <abstractions/base>
  #include <abstractions/nameservice>

  # Normal development work
  owner /home/** rw,
  /tmp/** rw,
  /usr/bin/* ix,
  /usr/lib/** r,

  # DENY access to ArmorIQ policy -- even as root
  deny /var/lib/armoriq/** rwkx,
  deny /usr/local/lib/armoriq/** wklx,

  # Allow read-only socket access (for enforcement queries)
  /var/run/armoriq/policyd.sock rw,

  # Prevent disabling AppArmor
  deny /sys/kernel/security/apparmor/** w,
  deny /etc/apparmor.d/** w,
}
```

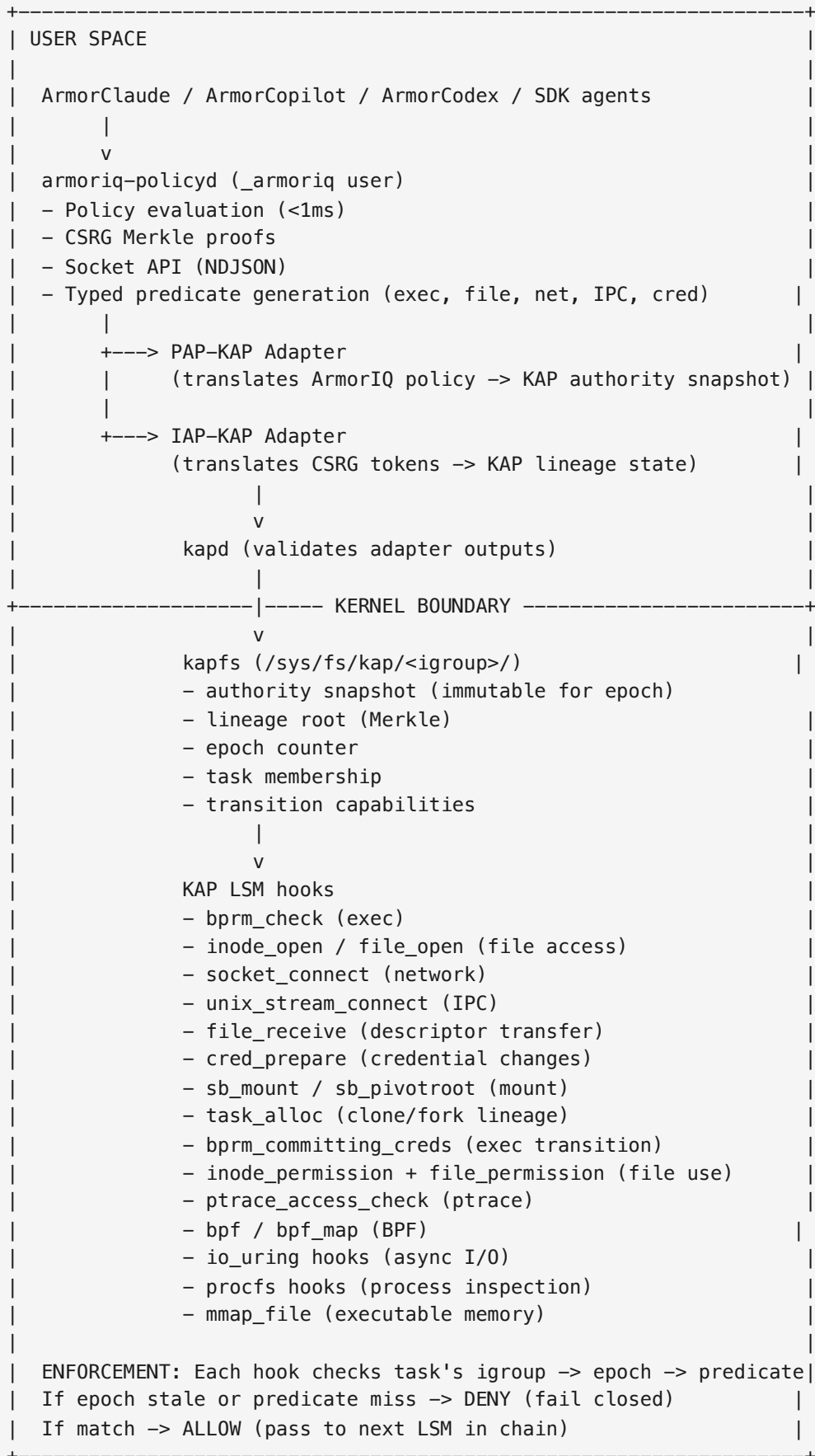
Deploying at v1.5 (macOS):

```
<!-- /Library/LaunchDaemons/io.armoriq.policyd.plist -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
  <key>Label</key>
  <string>io.armoriq.policyd</string>
  <key>ProgramArguments</key>
  <array>
    <string>/usr/local/lib/armoriq/policyd</string>
  </array>
  <key>UserName</key>
  <string>_armoriq</string>
  <key>GroupName</key>
  <string>_armoriq</string>
  <key>RunAtLoad</key>
  <true/>
  <key>KeepAlive</key>
  <true/>
  <key>SandboxProfile</key>
  <string>/usr/local/lib/armoriq/policyd.sb</string>
</dict>
</plist>
```

Part 7: Architecture WITH KAP (Future)

7.1 KAP Integration Architecture

When KAP matures (mainline inclusion or stable DKMS module), the integration adds a kernel enforcement layer below the existing daemon:



7.2 PAP-KAP Adapter for ArmorIQ

The adapter translates ArmorIQ policy statements into KAP typed predicates:

ArmorIQ Policy Statement =====		KAP Predicate(s) =====
permit tool Read	-->	file predicate: allow read on workspace
permit tool Grep	-->	exec predicate: allow /usr/bin/grep file predicate: allow read on workspace
forbid tool Write	-->	file predicate: deny write on workspace
forbid tool Bash	-->	exec predicate: deny /bin/bash, /bin/sh (or: no exec predicate = deny by omission)
require_approval for Bash	-->	(handled in user space -- KAP sees the post-approval predicate)
permit Bash when program in [ls]	-->	exec predicate: allow /bin/ls file predicate: allow read on /bin, /usr
deny MCP server evil-server	-->	ipc predicate: deny unix socket to evil-server net predicate: deny connect to evil-server
permit MCP server approved-server	-->	ipc predicate: allow unix socket to approved-server
workspace scope: /home/dev/project	-->	file predicate: allow r/w under /home/dev/project file predicate: deny everything else

The adapter follows KAP's conservative compilation rule: if an interface can't be fully mapped, it fails closed (no predicate installed = denied).

7.3 IAP-KAP Adapter for CSRG

ArmorIQ CSRG Event =====		KAP Lineage State =====
Policy created (initial)	-->	lineage root = Merkle(policy_v1) epoch = 1 igroup = armorclaudesession
/armor policy confirm <proposal>	-->	IAP Intent Commit new epoch = epoch + 1 new root = Merkle(policy_v2) transition: IntentCommit(e1->e2)
/armor policy add deny Bash	-->	IAP Trust Update transition capability installed authority narrows (Bash removed)
CSRG token verification	-->	lineage root comparison epoch consistency check
Policy tamper detected	-->	epoch revocation all igfds with old epoch fail closed daemon enters deny-all until restored

7.4 What Changes, What Stays

Component	Without KAP	With KAP	Changes?
ArmorClaude hooks	Thin client -> daemon	Same	No change
armoriq-policyd daemon	Policy eval + enforcement	Policy eval + adapter output	Minor: add predicate export
Socket protocol	NDJSON allow/deny	Same + predicate metadata	Additive
Policy JSON format	armor.policy.v1	Same	No change
CSRG Merkle tokens	Tamper detection	Tamper detection + lineage roots	Reuse existing
Enforcement location	Daemon (user space)	Daemon + kernel (both)	Additive layer
Sudo protection	MAC (SELinux/AppArmor)	KAP LSM hooks	KAP replaces MAC need
macOS support	Daemon + sandbox profile	Daemon + sandbox profile (no KAP)	No change on macOS

The key insight: nothing is thrown away. KAP is an additive layer.

Part 8: Security Deep Dive

8.1 Attack Vector Analysis (Current vs Daemon vs KAP)

#	Attack	Current (same user)	With daemon (separate user + MAC)	With KAP (kernel enforcement)
1	Write to policy.json via Edit/Write	Path guard (bypassable)	BLOCKED: Unix perms + MAC	BLOCKED: file predicate
2	Write to policy.json via Bash	Regex guard (bypassable)	BLOCKED: Unix perms + MAC	BLOCKED: file predicate
3	Import handler via Bash	Name regex (bypassable)	BLOCKED: code owned by _armoriq + MAC	BLOCKED: exec predicate
4	Base64-encode modifying script	No guard	BLOCKED: decoded script still can't write	BLOCKED: file predicate at kernel

#	Attack	Current (same user)	With daemon (separate user + MAC)	With KAP (kernel enforcement)
5	Kill enforcement daemon	Same user, works	BLOCKED: different user, EPERM; launchd restarts	BLOCKED: cred predicate
6	Read API key from credentials	Same user, works	BLOCKED: file 0600 + MAC	BLOCKED: file predicate
7	Modify daemon code/config	Same user, works	BLOCKED: code owned by _armoriq + MAC	BLOCKED: file predicate
8	Send fake policy mutation via socket	Can send anything	BLOCKED: requires auth:'user-session' + peer creds	BLOCKED: same + KAP IPC predicate
9	chmod policy files	Same user, works	BLOCKED: can't chmod files owned by other user + MAC	BLOCKED: cred predicate
10	Sudo to escalate	Requires password	BLOCKED IF: MAC + kernel lockdown active	BLOCKED: KAP enforces below root
11	Modify own config (CBSE)	No guard	BLOCKED: daemon enforces independently of agent config	BLOCKED: file predicate on config paths
12	Inject malicious MCP server	MCP registry in agent	BLOCKED: MCP registry in daemon	BLOCKED: IPC predicate
13	Cross-agent escalation	Possible	BLOCKED: each agent's policy daemon-protected	BLOCKED: separate igroups
14	Disable SELinux (setenforce 0)	N/A	BLOCKED IF: kernel lockdown active	N/A (KAP is the enforcement)
15	Prompt injection in tool output	No effect on /armor-policy	Same	Same
16	Social-engineer user	Human makes conscious decision	Same	Same
17	Authority drift (locally legal ops)	No detection	No detection (daemon checks policy, not lineage)	BLOCKED: KAP checks authority lineage

Score: - Current: 4/17 blocked - Daemon + MAC: **14/17 blocked** (social engineering, prompt injection into user, authority drift remain) - Daemon + MAC + KAP: **15/17 blocked** (only social engineering and deliberate human decision remain)

Attack #17 (authority drift) is the unique KAP contribution – no other mechanism detects locally-legal operations whose authority drifted beyond the task.

8.2 The Socket Authentication Model

The daemon socket is accessible to any user in the staff group (mode 0660). Claude CAN connect and send messages. Here's why that's still secure:

Read operations (no auth needed): - `type: 'hook'` -> PreToolUse enforcement queries - `type: 'policy_read'` -> list policies - `type: 'ping'` -> health check

These are fine for Claude to call. Getting policy decisions is the whole point.

Write operations (auth required): The `auth: 'user-session'` tag is set by the UserPromptSubmit hook handler. This hook only fires on human terminal input.

Defense in depth for socket auth: - Level 1: auth tag presence (filters casual attempts) - Level 2: Peer credential check (SO_PEERCRED on Linux / LOCAL_PEERCRED on macOS) – daemon verifies the connecting PID - Level 3: Process tree check – verify PID is a child of Claude Code's hook spawn - Level 4: Nonce verification – daemon issues a nonce at session start, hook must present it with mutations

8.3 CBSE Protection Analysis

ArmorIQ's daemon architecture provides structural immunity to CBSE:

CBSE Attack Flow (vulnerable systems):

Agent -> modifies `.mcp.json` -> next session loads malicious MCP -> code execution

ArmorIQ Defense:

Agent -> attempts to modify config -> irrelevant

Because: enforcement happens in daemon, NOT in agent config

Daemon reads policy from `/var/lib/armoriq/` (owned by `_armoriq`)

Agent's `.mcp.json` modifications have no effect on enforcement

MCP registry lives in daemon, not in agent config files

8.4 Failure Modes

Scenario	Behavior	Why correct
Daemon not installed	ArmorClaude enters observe-only mode	User chose not to install; docs explain trade-off
Daemon crashes	launchd/systemd restarts; hooks fail-closed (deny-all) during gap	Better to block briefly than allow everything

Scenario	Behavior	Why correct
Socket file deleted	Daemon recreates on restart; hooks fail-closed	Transient, auto-recovers
Backend unreachable	Local enforcement continues; audit WAL queues	Offline-first design
Policy file corrupted	Crypto digest mismatch -> deny-all + alert	Tamper detection at daemon level
SELinux disabled	If kernel lockdown active: can't disable. If not: daemon + DAC still enforce	Defense in depth
KAP module unloaded (future)	Daemon enforcement continues (user-space layer intact)	KAP is additive, not replacing
OPA unreachable	Circuit breaker -> falls back to local JSON	Graceful degradation

8.5 Comparison: ArmorIQ vs Industry

Capability	OPA/Styra	Cerbos	Permit.io	Invariant	Guardian Shell	ArmorIQ (proposed)
Separate trust domain	Sidecar	Sidecar/service	Sidecar PDP	Gateway proxy	Kernel enforcement	System user daemon + MAC + (future) KAP
Stops root/sudo	No (same container)	No	No	No	Partial (Landlock)	YES (MAC + lockdown; future: KAP)
CBSE protection	No	No	No	No	Partial	YES (architectural)
Authority lineage	No	No	No	No	No	Future (KAP)
Evaluation latency	Sub-ms	Sub-ms	1-5ms	Near-zero	Low	<5ms (Unix socket)
Offline support	Yes (cached)	Yes (local)	Partial	No	Yes	Yes (local JSON)
AI agent awareness	No	No	No	Yes	Yes	Yes
Multi-product	Generic	Generic	Generic	AI agents only	AI agents only	AI tools specific
Human-only mutation	N/A	N/A (git PR)	N/A	N/A	Partial	UserPromptSubmit hook

Capability	OPA/Styra	Cerbos	Permit.io	Invariant	Guardian Shell	ArmorIQ (proposed)
Crypto tamper detection	Bundle signing	No	No	No	No	CSRG Merkle proofs
Cross-platform	Yes	Yes	Yes	Yes	Linux only	macOS + Linux

Part 9: Implementation Strategy

9.1 Phase Overview (Revised)

Phase	What	Effort	Delivers	Sudo Protection
Phase 0 (NOW)	Ship current architecture as v0.3	1 week	Working policy, 195 tests, immediate value	None (same user)
Phase 1	Create armoriq-policyd repo, extract daemon	2 weeks	Standalone daemon with socket API	DAC (separate user)
Phase 2	System service installer (launchd + systemd)	2 weeks	OS-level isolation, _armoriq user	DAC + sudo stripping + capabilities
Phase 3	ArmorClaude thin client migration	1 week	ArmorClaude becomes ~200 lines	(inherits Phase 2)
Phase 4	Socket auth hardening + MAC profiles	2 weeks	Peer creds, nonce; SELinux/AppArmor/macOS sandbox	DAC + MAC + kernel lockdown
Phase 5	Cross-product client library	2 weeks	@armoriq/policy-client npm + Python packages	(inherits Phase 4)
Phase 6	eBPF/Tetragon integration (optional)	2 weeks	Runtime authority monitoring	Kernel-level observability
Phase 7 (FUTURE)	KAP integration	4 weeks	PAP-KAP + IAP-KAP adapters, kernel enforcement	Full kernel enforcement

9.2 What Goes Where

armoriq-policyd (new repo) – the daemon and all enforcement logic:

```

armoriq-policyd/
+-- src/
|   +-- daemon.mjs           # Socket server, message routing
|   +-- engine.mjs          # Enforcement handlers
|   +-- armor-policy-commands.mjs # Policy CRUD
|   +-- policy.mjs           # Policy evaluation
|   +-- policy-templates.mjs  # Template system
|   +-- policy-profiles.mjs    # Profile management
|   +-- policy-compiler.mjs    # Compile to OPA / KAP predicates <-- NEW
|   +-- predicate-generator.mjs # Typed predicate output (KAP-ready) <-- NE
|   +-- tool-registry.mjs     # Tool allowlists
|   +-- opa-client.mjs        # OPA PDP client
|   +-- crypto-policy.mjs     # CSRG Merkle proofs
|   +-- backend-client.mjs    # Cloud sync
|   +-- runtime-state.mjs     # Session state
|   +-- audit-wal.mjs         # Write-ahead log
|   +-- fs-store.mjs          # File storage
|   +-- config.mjs            # Daemon config (/var/lib/armoriq/)
+-- install/
|   +-- install.sh            # Cross-platform installer
|   +-- io.armoriq.policyd.plist # macOS launchd
|   +-- armoriq-policyd.service # Linux systemd
|   +-- armoriq.te             # SELinux policy module
|   +-- armoriq.apparmor       # AppArmor profile
|   +-- armoriq-agent.sb       # macOS sandbox profile
+-- tests/
+-- package.json

```

armorclaudes (existing repo) – thin client only:

```

armorclaudes/
+-- hooks/hooks.json
+-- skills/armor-policy/SKILL.md
+-- scripts/
|   +-- bootstrap.mjs
|   +-- hook-router.mjs      # THIN CLIENT: socket -> daemon
|   +-- policy-mcp.mjs      # MCP server (reads via daemon)
|   +-- lib/
|       +-- daemon-client.mjs # Socket connect + NDJSON
+-- package.json

```

9.3 Installer Experience

```
# Developer runs:
curl -fsSL https://install.armoriq.ai/policyd | sudo bash

# What happens:
# 1. Creates _armoriq system user (if not exists)
# 2. Creates /var/lib/armoriq/ with correct ownership (0700)
# 3. Installs daemon to /usr/local/lib/armoriq/
# 4. Installs launchd plist (macOS) or systemd unit (Linux)
# 5. Sets immutable attributes on policy files (Linux)
# 6. Installs SELinux/AppArmor module if available
# 7. Starts the daemon
# 8. Adds current user to staff group (for socket access)
# 9. Prints: 'ArmorIQ Policy Daemon installed. Hardening level:
           [standard|mac|full]'

# Hardening levels:
# standard: separate user + DAC + sudo stripping
# mac: standard + SELinux/AppArmor + kernel lockdown check
# full: mac + immutable attrs + auditd + namespace isolation

# Verify:
armoriq-policyd status
# -> ArmorIQ Policy Daemon v2.0.0 | running | uptime: 5s | hardening: mac

# Or via Homebrew (macOS):
brew install armoriq-policyd
```

Part 10: Open Questions

1. **Should the daemon support Windows?** Named pipes instead of Unix sockets, Windows service instead of launchd/systemd, Windows Defender Application Control instead of SELinux.
2. **Homebrew formula?** `brew install armoriq-policyd` avoids curl-pipe-sudo friction. Signed with Apple Developer ID.
3. **Should we build an admin dashboard?** Web UI for policy management, complementing /armor-policy CLI.
4. **How should org-wide policy distribution work?** Backend pushes to daemons? Daemons poll? Git-based like Cerbos?
5. **Should we support 'break glass'?** Emergency override requiring physical MFA token?
6. **Landlock from Node.js?** Can we use Landlock self-sandbox from a Node.js native addon, or must the launcher be compiled?

7. **armorIQ-policyd API format?** Should policy eval return typed predicates now (KAP-ready) or just allow/deny with predicate metadata in a side channel?
8. **KAP upstream timeline?** Track LKML discussion. Consider DKMS module for enterprise early adopters.
9. **eBPF/Tetragon integration scope?** Runtime monitoring only, or active enforcement? Integration with Cilium for network-level agent authority?

Part 11: Risk Assessment

Risk	Impact	Likelihood	Mitigation
sudo installation friction reduces adoption	High	Medium	Homebrew formula; observe-only mode without daemon; clear docs
macOS Gatekeeper blocks unsigned daemon	Medium	High	Sign with Apple Developer ID; distribute via Homebrew
Socket permission misconfiguration	High	Low	Installer validates; daemon refuses to start if insecure
SELinux/AppArmor misconfiguration leaves gap	High	Medium	Installer auto-detects and configures; hardening-level output
Users lose policy when daemon crashes	Medium	Low	launchd/systemd auto-restart; WAL for audit durability
Version mismatch (client vs daemon)	Medium	Medium	Version handshake in ping; client warns on mismatch
Breaking change in Claude Code hook system	High	Low	Hooks are Claude Code's documented public API
CBSE attack on ArmorIQ itself	High	Low	Daemon owns all config; agent config changes don't affect enforcement
Competitor ships similar product faster	Medium	Medium	Ship v0.3 immediately; daemon is v1.0 upgrade
KAP never goes upstream	Medium	Medium	Without-KAP stack is complete and sufficient;

Risk	Impact	Likelihood	Mitigation
			KAP is additive bonus
Kernel lockdown not enabled on customer systems	Medium	High	Document as recommendation; installer checks and warns
New LLM capabilities bypass current defenses	High	Medium	Layered defense means no single bypass is total; monitor research

Appendix A: Research Sources

All findings in this report were verified through adversarial verification (3-vote panels, 2/3 refute to kill):

1. Datadog Security Labs – Container Security Fundamentals Part 5 (MAC vs DAC analysis)
2. Red Hat Article 6964380 – SELinux as Security Pillar (CVE mitigation evidence)
3. man7.org – kernel_lockdown(7) (root constraint mechanisms)
4. arxiv:2605.03213v1 – Imperial College London TEE Survey, May 2026 (enclave limitations)
5. Tetragon project – eBPF enforcement capabilities and limitations
6. Johann Rehberger (wunderwuzzi) – Cross-Agent Privilege Escalation demonstration
7. Cymulate Research Labs – CBSE (Configuration-Based Sandbox Escape) discovery
8. UK AI Safety Institute – SandboxEscapeBench, March 2026 (container escape rates)
9. arxiv:2601.11893v1 – LLM Agent Privilege Escalation Taxonomy
10. KAP Paper – Bhardwaj, Georgia Tech (Kernel Assurance Plane design and evaluation)

Appendix B: Glossary

Term	Definition
DAC	Discretionary Access Control – Unix file permissions. Owner-controlled. Root bypasses.
MAC	Mandatory Access Control – SELinux/AppArmor. System-controlled. Root is subject to policy.
PDP	Policy Decision Point – evaluates allow/deny queries
PAP	Policy Administration Point – manages policy CRUD
KAP	Kernel Assurance Plane – kernel-level authority lineage enforcement

Term	Definition
igroup	Intent Lineage Group – KAP's unit of isolation
igfd	igroup file descriptor – handle carrying authority epoch
CSRG	Cryptographic State Reference Graph – ArmorIQ's Merkle-based tamper detection
CBSE	Configuration-Based Sandbox Escape – attack via modifying agent config files
NDJSON	Newline-delimited JSON – wire protocol for daemon socket
TEE	Trusted Execution Environment – hardware-isolated execution
SGX	Software Guard Extensions – Intel's process-level enclave (deprecated on consumer CPUs)
SEV-SNP	Secure Encrypted Virtualization – Secure Nested Paging – AMD's VM-level encryption
TDX	Trust Domain Extensions – Intel's VM-level isolation